

Buổi

8

YOEDU · Spring Boot 4

Điểm danh & Thông báo

💡 Một service method ghi 2 bảng — đây là lúc @Transactional cứu bạn.



Mục tiêu buổi học

- Điểm danh học viên theo ngày
- Service tự sinh notification khi học viên vắng
- Hiểu @Transactional bảo vệ dữ liệu như thế nào



Kết quả mong đợi

- Gọi được POST attendance → notification tự sinh
- Giải thích được tại sao cần @Transactional
- Debug được lỗi trùng attendance hoặc thiếu notification

Buổi 8 · Bạn đã biết điều này — nhưng hôm nay khác ở đâu?



Trong các bài trước

- Controller gọi Service gọi Repository — bạn đã biết
- Ghi 1 bảng: save() xong là xong
- Lỗi giữa chừng? Tự catch, tự xử lý — rất phức tạp
- Transaction? Phải dùng JDBC hoặc config XML dài dòng



Spring Boot thực tế

- Một method ghi cùng lúc 2 bảng: attendance + notification
- Chỉ cần @Transactional — Spring lo phần còn lại
- Lỗi giữa chừng? Cả 2 bảng tự rollback, DB không bị lệch
- Service layer giờ là trái tim của toàn bộ nghiệp vụ

⚡ Điểm mới của buổi này: bạn sẽ thấy @Transactional làm gì mà try/catch không làm được.

Buổi 8 · Các file quan trọng của buổi học này

#	Tên File	Mục đích khi mở	Điểm cần nói với SV
1	Attendance.java	Entity lưu bản ghi điểm danh	<i>Chỉ ra field status — đây là trigger sinh notification</i>
2	Notification.java	Entity thông báo gửi phụ huynh	<i>Xem NotificationType để thấy hệ thống mở rộng được</i>
3	AttendanceCreateRequest.java	DTO input cho thao tác điểm danh	<i>Hỏi SV: tại sao không truyền thẳng Entity vào Controller?</i>
4	AttendanceService.java	Logic điểm danh + tạo notification	<i>File quan trọng nhất — mở cuối cùng sau khi SV đã hiểu flow</i>
5	AttendanceController.java	Endpoint REST	<i>Controller mỏng: chỉ validate input, gọi service, trả response</i>



Tip: Mở từng file theo thứ tự — giải thích MỤC ĐÍCH trước, đọc code sau.

Buổi 8 · Workflow A — Điểm danh học viên

Vẽ / giải thích workflow này TRƯỚC khi mở IDE.

Workflow A



⚡ Điểm cần nhấn mạnh

Cả bước 4 và 5 nằm trong cùng một @Transactional method — nếu bước 5 lỗi, bước 4 sẽ bị rollback tự động.

Buổi 8 · @Transactional — thứ trường chưa dạy bạn

Đây là lý do tại sao Spring Boot an toàn hơn code JDBC/Servlet thuần khi ghi nhiều bảng.

✗ Không có @Transactional

```
attendanceRepo.save(attendance);  
// ✅ Đã ghi vào DB  
  
// ✨ Đây lỗi bất ngờ  
notificationRepo.save(noti);
```

- attendance đã lưu
- notification KHÔNG lưu
- Dữ liệu bị lệch, không ai biết!



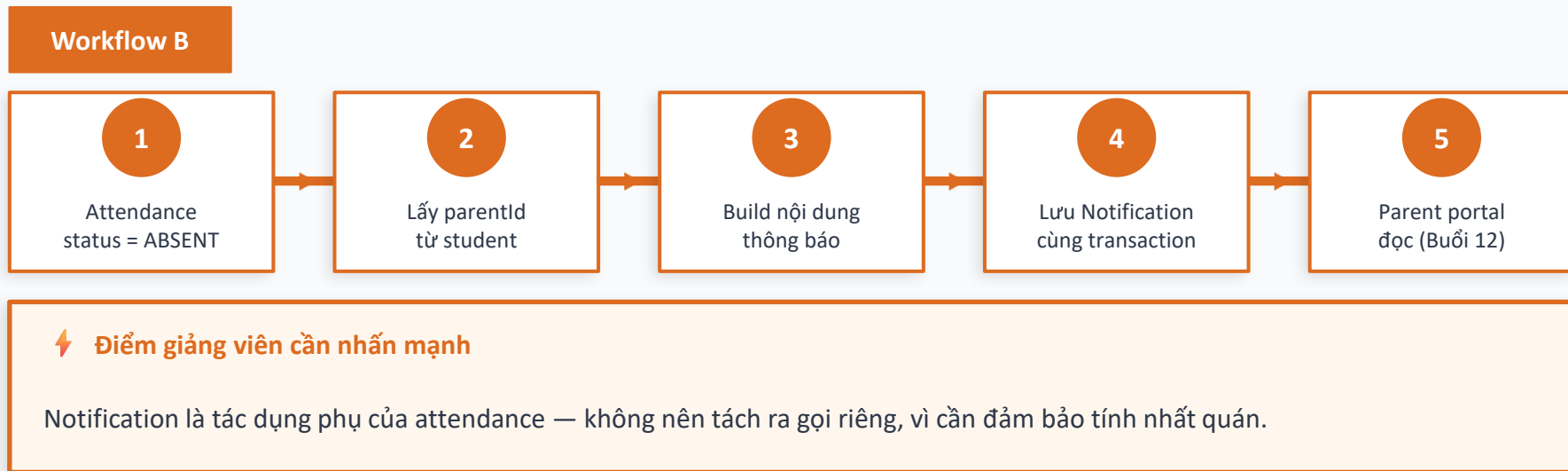
✅ Có @Transactional

```
@Transactional  
public void markAttendance() {  
    attendanceRepo.save(a);  
    notificationRepo.save(n);  
} // lỗi → rollback cả 2!
```

- Nếu lỗi giữa chừng:
- CẢ HAI đều bị hủy
- DB sạch, không lệch dữ liệu ✅

🎓 Câu hỏi kích thích: Nếu không có @Transactional, attendance đã ghi xong — làm sao bạn rollback nó?

Buổi 8 · Workflow B — Thông báo vắng học



Buổi 8 · Lỗi thực tế & cách debug

Đây là những lỗi thường gặp — biết trước để không mất cả tiếng ngồi Google.

DataIntegrityViolationException

 **Nguyên nhân:** Gọi POST attendance 2 lần cùng class + student + date (thiếu unique constraint)

 **Cách fix:** Thêm @UniqueConstraint hoặc kiểm tra trong service trước khi save()

notification sinh ra nhưng attendance không lưu

 **Nguyên nhân:** Thiếu @Transactional — 2 save() chạy trong 2 transaction riêng biệt

 **Cách fix:** Thêm @Transactional vào service method — cả 2 save() dùng chung 1 transaction

NullPointerException trong NotificationService

 **Nguyên nhân:** Student chưa có parent liên kết — getParent() trả về null

 **Cách fix:** Kiểm tra null trước khi build notification, hoặc dùng Optional<Parent>

Buổi 8 · Những phần code quan trọng

Entity / Enum

- Attendance
- Notification
- AttendanceStatus
- (PRESENT/ABSENT/LATE)
- NotificationType
- NotificationRecipientType

→ Mô tả cấu trúc dữ liệu lưu vào DB

Service

- AttendanceService
- @Transactional mark()
- → save attendance
- → if ABSENT: save noti

→ Giữ toàn bộ logic nghiệp vụ

Controller

- AttendanceController
- POST /api/attendances
- GET /api/attendances
- /class/{classId}

→ Chỉ nhận request & gọi service

 Trong file `AttendanceService.java` — `@Transactional` đặt ở method chứa cả 2 `save()`.

Buổi 8 · Các endpoint cần viết

POST

/api/attendances

tạo + sinh notification

GET

/api/attendances/class/{classId}

xem danh sách lớp



Trình tự

- Happy path: POST status=PRESENT → chỉ tạo attendance
- POST status=ABSENT → kiểm tra DB có cả attendance + notification
- POST trùng → phải ra lỗi rõ ràng, không im lặng



Điểm cần quan sát

- Response body trả về gì? (chỉ attendance hay cả notification?)
- @Transactional có hiện trong log không?
- Nếu tắt DB giữa chừng, cái gì rollback trước?



Phải hoàn thành trong buổi

1. Thêm @Transactional vào service method ghi nhiều bảng
2. Unique theo class + student + date (không ghi trùng)
3. Tách rõ: attendance logic và notification logic trong cùng method
4. Ghi nhận recordedBy để phục vụ audit sau này

8

Bài tập cuối buổi & Đầu ra



Bài tập giao cuối buổi

- Thêm trạng thái LATE (đi trễ) — notification cho LATE khác ABSENT như thế nào?
- Viết unit test cho AttendanceService: trường hợp ABSENT và PRESENT
- Thử bỏ @Transactional, chạy lại POST, quan sát sự khác biệt trong DB



Sinh viên cần

- Giải thích được @Transactional bằng lời của mình
- Xử lý được nghiệp vụ nhiều bước với tính nhất quán dữ liệu
- Biết cách debug lỗi trùng hoặc thiếu dữ liệu sau khi ghi

"Một annotation — hai bảng an toàn. Đó là Spring Boot."